

Chapter 1 :The Bayesian paradigm for statistical Inference

Python Lab sessions

Exercise 1

```
In [8]: import matplotlib.pyplot as plt
import numpy as np
from scipy import stats

mu_params = [-1, 0, 1]
sd_params = [0.5, 1, 1.5]
x = np.linspace(-7, 7, 200)

f, ax = plt.subplots(len(mu_params), len(sd_params), sharex=True, sharey=True, figsize=(10, 10))

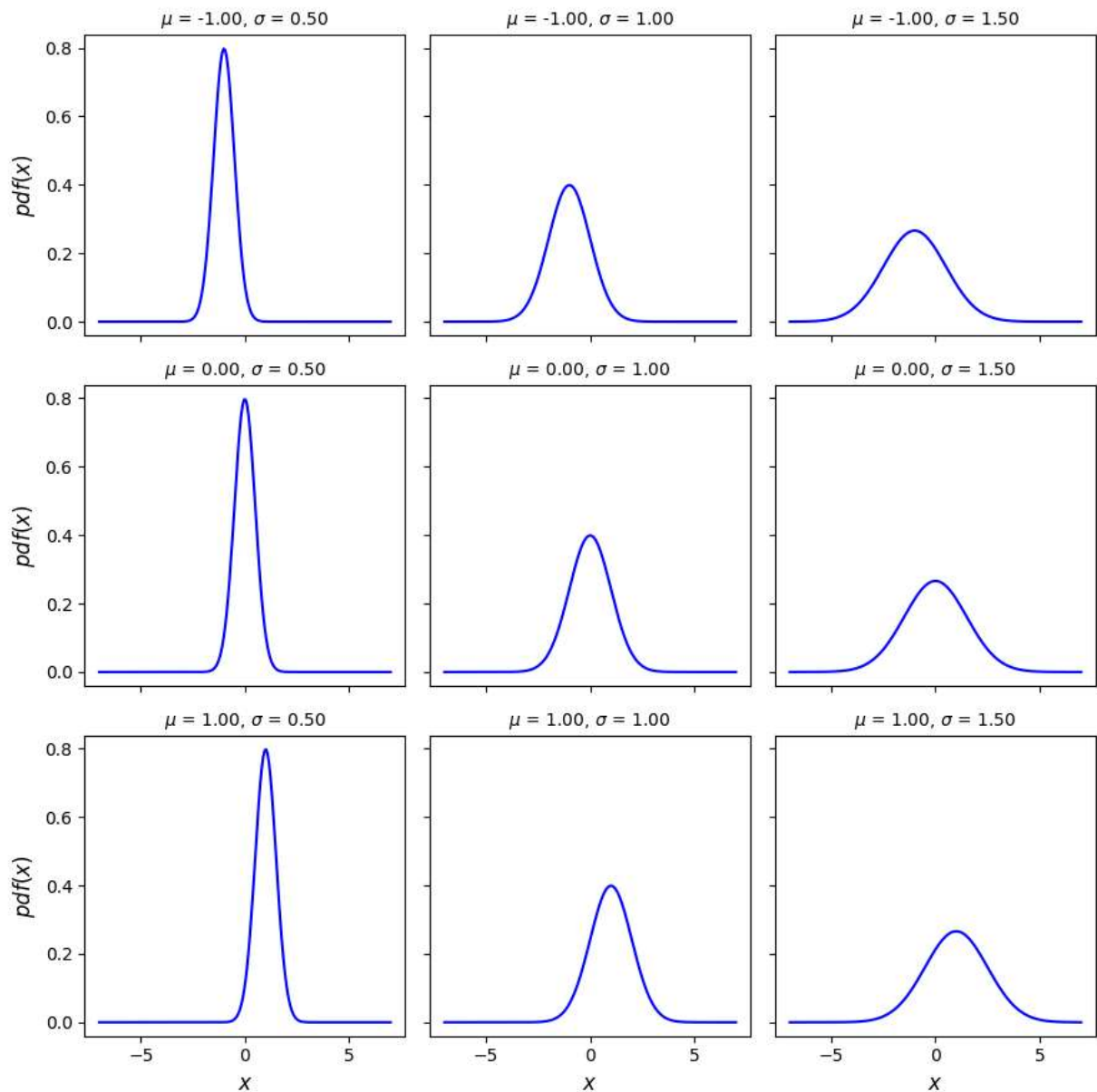
for i in range(len(mu_params)):
    for j in range(len(sd_params)):
        mu = mu_params[i]
        sd = sd_params[j]

        y = stats.norm(mu, sd).pdf(x)
        ax[i, j].plot(x, y, color='b')
        ax[i, j].set_title(rf'$\mu$ = {mu:.2f}, $\sigma$ = {sd:.2f}', fontsize=10)

# Axis Labels (only outer edges)
for i in range(len(mu_params)):
    ax[i, 0].set_ylabel(r'$pdf(x)$', fontsize=12)

for j in range(len(sd_params)):
    ax[-1, j].set_xlabel(r'$x$', fontsize=12)

plt.tight_layout()
plt.show()
```



Exercise 2

```
In [9]: import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

n_params = [1, 2, 4]
p_params = [0.25, 0.5, 0.75]
x = np.arange(0, max(n_params) + 1)

f, ax = plt.subplots(len(n_params), len(p_params), sharex=True, sharey=True, figsize=(10, 10))

for i in range(len(n_params)):
    for j in range(len(p_params)):
        n = n_params[i]
        p = p_params[j]
```

```

y = stats.binom(n=n, p=p).pmf(x)

# Plot as vertical lines (discrete distribution)
ax[i, j].vlines(x, 0, y, colors='b', lw=4)
ax[i, j].set_ylim(0, 1)
ax[i, j].set_title(f"n = {n}, p = {p:.2f}", fontsize=10)

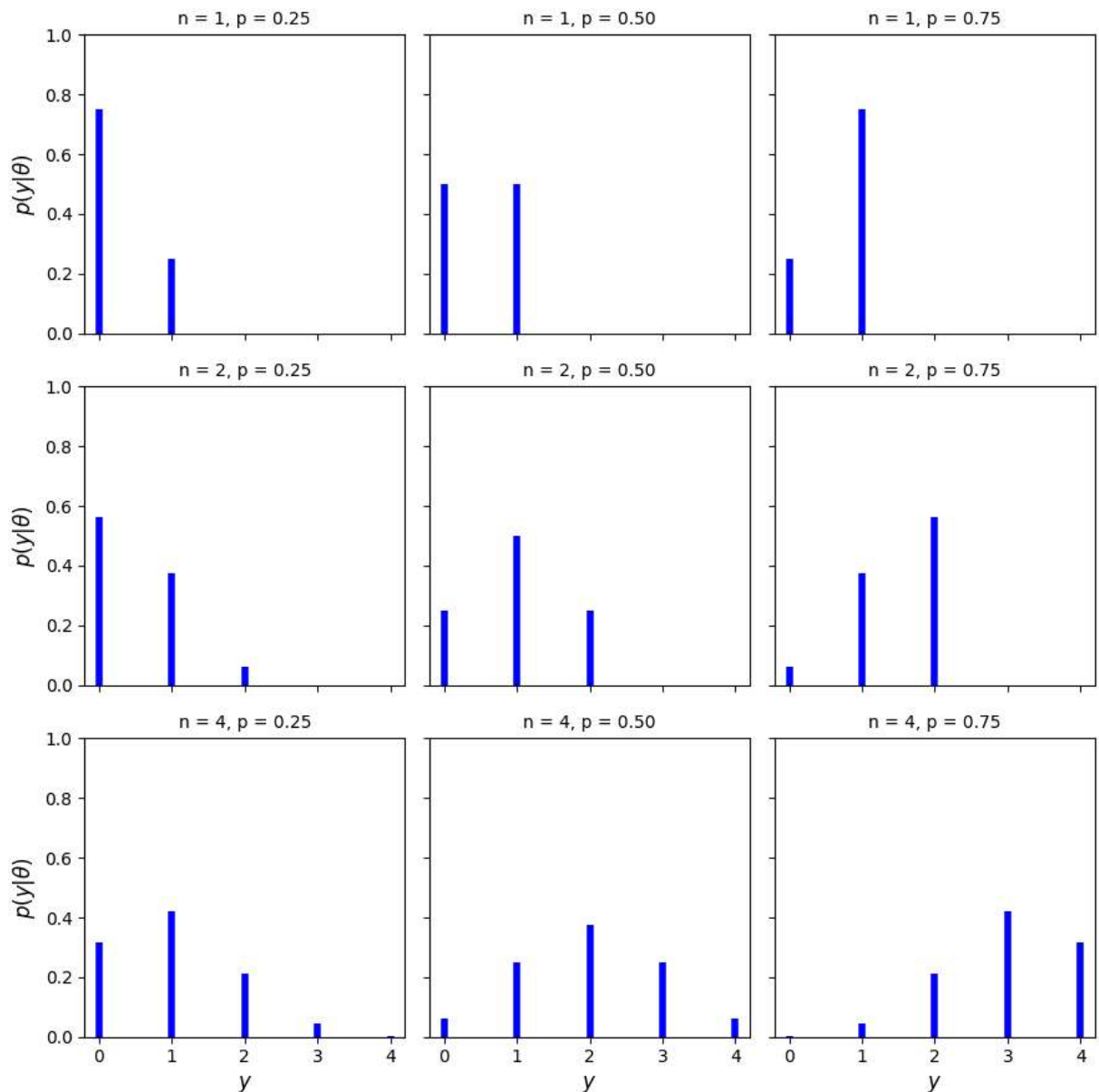
# Show only valid x values for each n
ax[i, j].set_xticks(np.arange(0, n + 1))

# Axis Labels (only outer edges)
for i in range(len(n_params)):
    ax[i, 0].set_ylabel(r'$p(y|\theta)$', fontsize=12)

for j in range(len(p_params)):
    ax[-1, j].set_xlabel(r'$y$', fontsize=12)

plt.tight_layout()
plt.show()

```



Exercise 3

```
In [10]: import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

params = [0.5, 1, 2, 3]
x = np.linspace(0, 1, 100)

f, ax = plt.subplots(len(params), len(params), sharex=True, sharey=True, figsize=(10, 10))

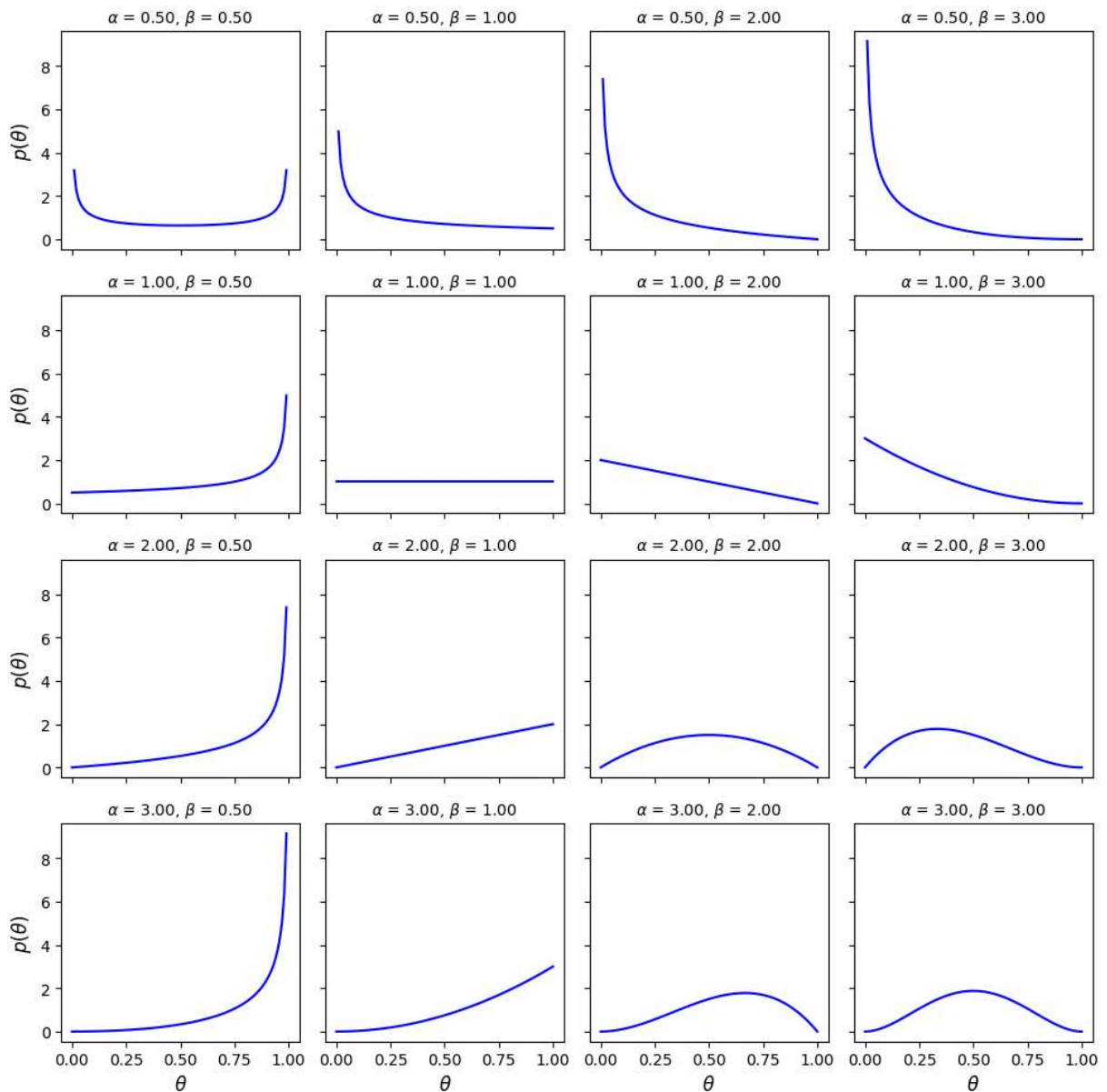
for i in range(len(params)):
    for j in range(len(params)):
        a = params[i]
        b = params[j]

        y = stats.beta(a, b).pdf(x)
        ax[i, j].plot(x, y, color='b')
        ax[i, j].set_title(rf'$\alpha$ = {a:.2f}, $\beta$ = {b:.2f}', fontsize=10)

# Axis Labels (only on outer edges for clarity)
for i in range(len(params)):
    ax[i, 0].set_ylabel(r'$p(\theta)$', fontsize=12)

for j in range(len(params)):
    ax[-1, j].set_xlabel(r'$\theta$', fontsize=12)

plt.tight_layout()
plt.show()
```



Exercise 4

```
In [11]: import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

theta_real = 0.35
trials = [0, 1, 2, 3, 4, 8, 16, 32, 50, 150]
data = [0, 1, 1, 1, 1, 4, 6, 9, 13, 48]
beta_params = [(1, 1), (0.5, 0.5), (20, 20)]

dist = stats.beta
x = np.linspace(0, 1, 100)

plt.figure(figsize=(10, 10))

for idx, N in enumerate(trials):
    if idx == 0:
```

```
plt.subplot(4, 3, 2)
else:
    plt.subplot(4, 3, idx + 3)

y = data[idx]

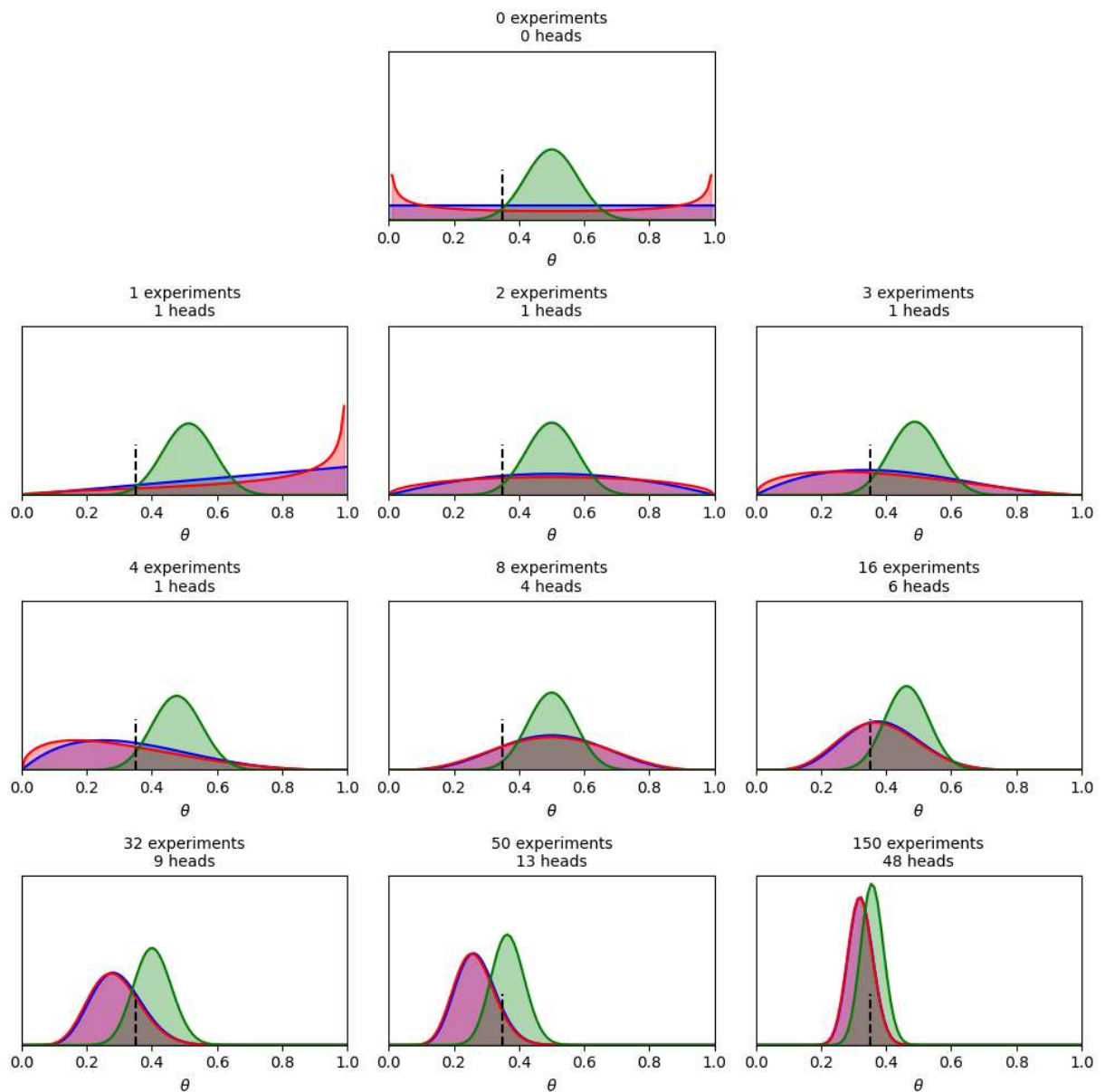
# Plot posterior distributions
for (a_prior, b_prior), c in zip(beta_params, ('b', 'r', 'g')):
    p_theta_given_y = dist.pdf(x, a_prior + y, b_prior + N - y)
    plt.plot(x, p_theta_given_y, c)
    plt.fill_between(x, 0, p_theta_given_y, color=c, alpha=0.3)

# True theta line
plt.axvline(theta_real, ymax=0.3, color='k', linestyle='--')
plt.title(f"{N} experiments\n{y} heads", fontsize=10)

plt.xlim(0, 1)
plt.ylim(0, 12)
plt.xlabel(r'$\theta$')

# for cleaner look
plt.gca().axes.get_yaxis().set_visible(False)

plt.tight_layout()
plt.show()
```

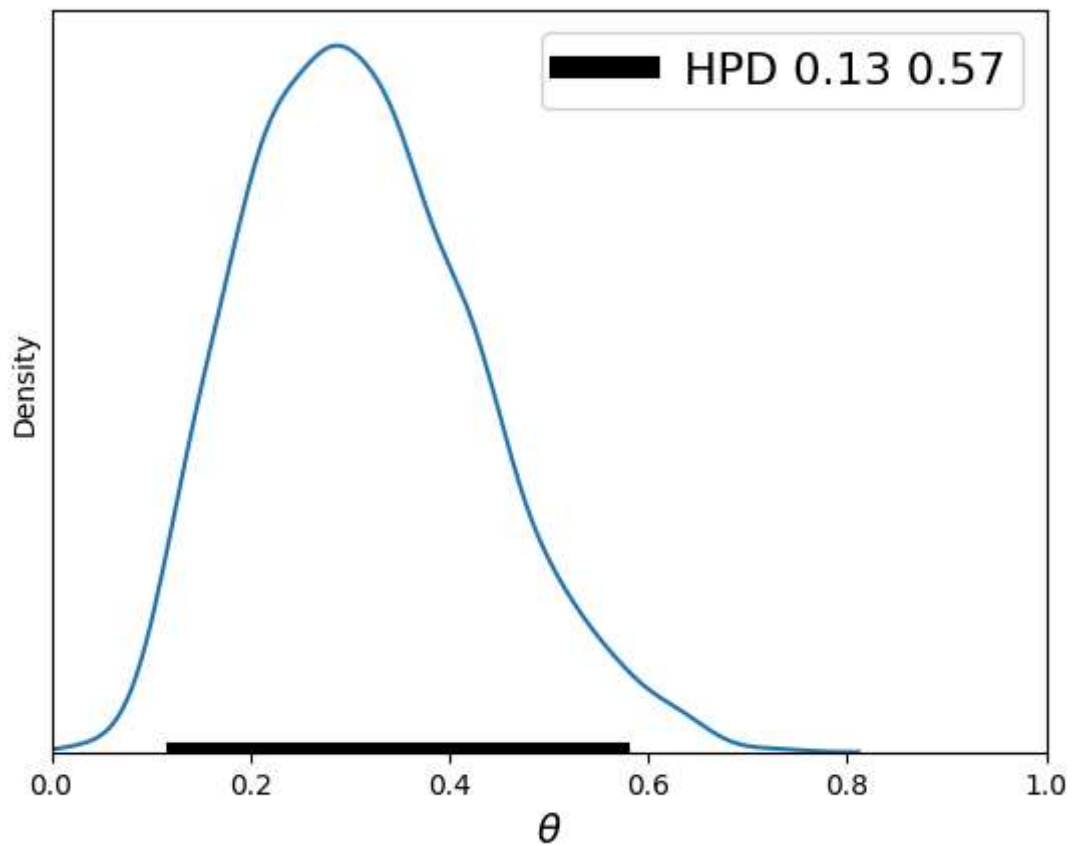


Exercise 5

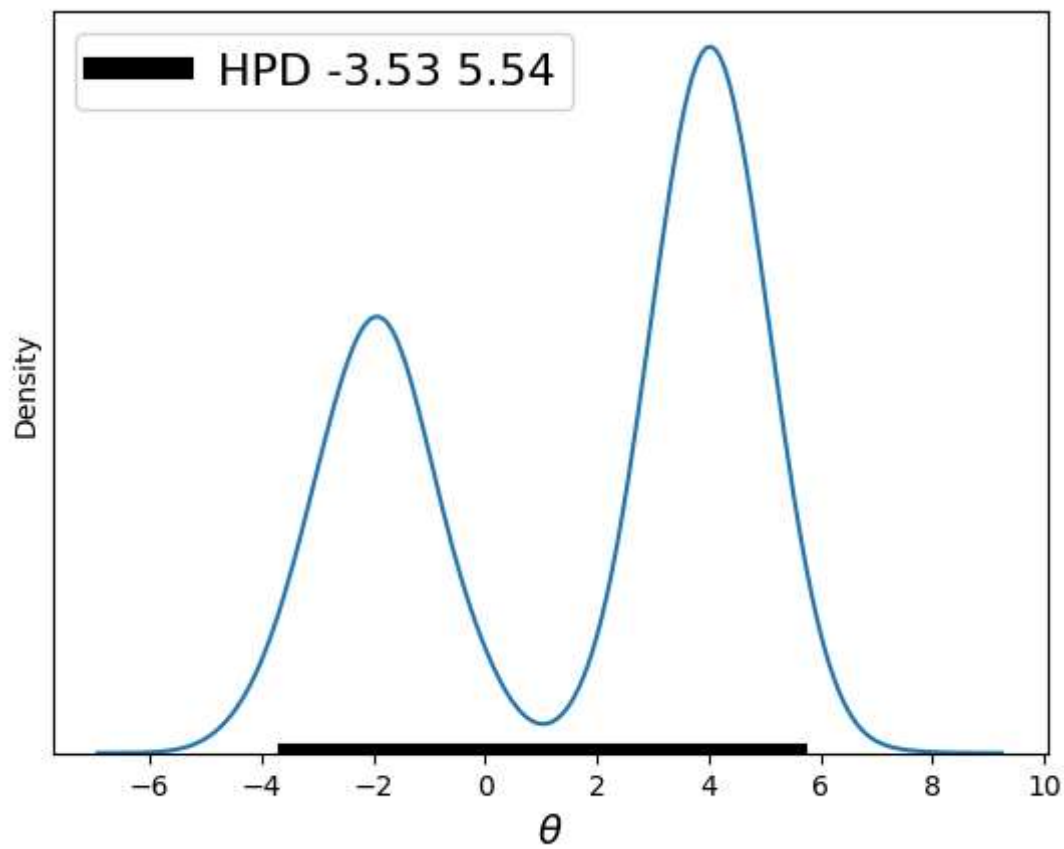
```
In [14]: import seaborn as sns

def naive_hpd(post):
    sns.kdeplot(post)
    HPD = np.percentile(post, [2.5, 97.5])
    plt.plot(HPD, [0, 0], label='HPD {:.2f} {:.2f}'.format(*HPD),
             linewidth=8, color='k')
    plt.legend(fontsize=16);
    plt.xlabel(r'$\theta$', fontsize=14)
    plt.gca().axes.get_yaxis().set_ticks([])
    np.random.seed(1)
    post = stats.beta.rvs(5, 11, size=1000)
    naive_hpd(post)
    plt.xlim(0, 1)
```

Out[14]: (0.0, 1.0)



```
In [15]: np.random.seed(1)
gauss_a = stats.norm.rvs(loc=4, scale=0.9, size=3000)
gauss_b = stats.norm.rvs(loc=-2, scale=1, size=2000)
mix_norm = np.concatenate((gauss_a, gauss_b))
naive_hpd(mix_norm)
```

In []: